

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 2

Выполнила:

Космач Мария

Группа 3439

Проверил:

Добряков Д. И.

Санкт-Петербург

2025 г.

Задача

По выбранному варианту необходимо будет реализовать RESTful API средствами express + typescript (используя ранее написанный boilerplate).

Ход работы

Проект был написан на основе boilerplate, реализованного в рамках ЛР1.

На рисунке 1 продемонстрирована структура проекта.

На рисунке 2 продемонстрирован AdvertisementController, который работает с AdvertisementService, продемонстрированного на рисунке 3.

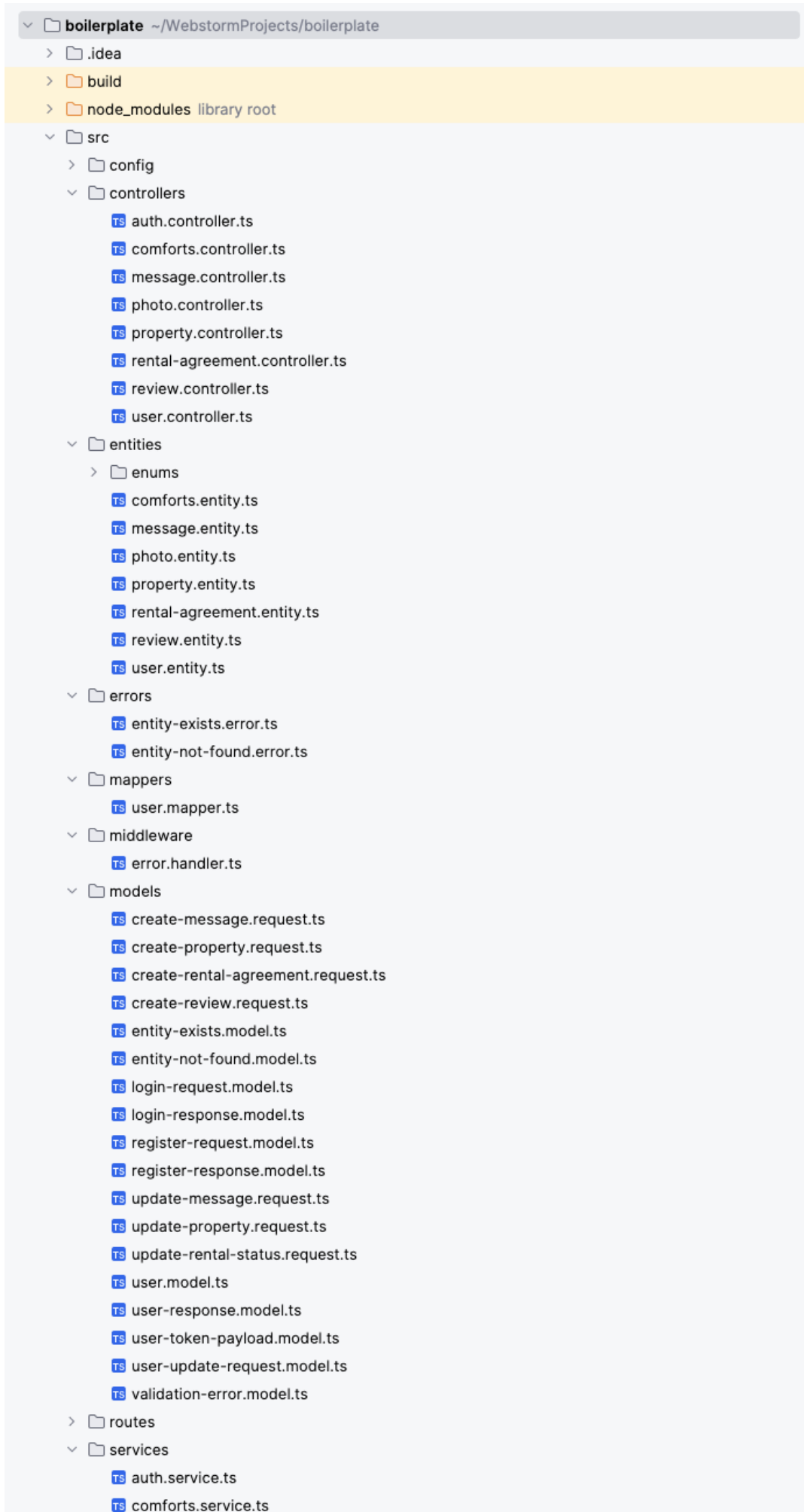


Рисунок 1 – Структура проекта

```

@Route("properties") Show usages
@Tags("Properties")
export class PropertyController extends Controller {
    @Get() Show usages
    @SuccessResponse("200", "Ok")
    public async getAllProperties(): Promise<PropertyEntity[]> {
        return propertyService.getAll();
    }

    @Get("{id}") Show usages
    @SuccessResponse("200", "Ok")
    @Response<EntityNotFoundErrorResponse>(404, "Property not found")
    public async getPropertyById(@Path() id: number): Promise<PropertyEntity> {
        return propertyService.getById(id);
    }

    @Post() Show usages
    @SuccessResponse("201", "Created")
    @Security("jwt")
    public async create(@Body() body: CreatePropertyRequest, @Request() req: any): Promise<PropertyEntity> {
        return propertyService.create(body, req.user.id);
    }

    @Put("{id}") Show usages
    @SuccessResponse("200", "Updated")
    @Response<EntityNotFoundErrorResponse>(404, "Property not found")
    public async update(
        @Path() id: number,
        @Body() body: UpdatePropertyRequest
    ): Promise<PropertyEntity> {
        return propertyService.update(id, body);
    }

    @Delete("{id}") Show usages
    @SuccessResponse("204", "Deleted")
    @Response<EntityNotFoundErrorResponse>(404, "Property not found")
    public async delete(@Path() id: number): Promise<void> {
        await propertyService.delete(id);
        return;
    }
}

```

Рисунок 2 - AdvertisementController

```

class PropertyService { Show usages
  private repo : Repository<PropertyEntity> = dataSource.getRepository(PropertyEntity);
  private userRepo : Repository<UserEntity> = dataSource.getRepository(UserEntity);

  public async getAll(): Promise<PropertyEntity[]> { Show usages
    return this.repo.find();
  }

  public async getById(id: number): Promise<PropertyEntity> { Show usages
    const entity : PropertyEntity = await this.repo.findOne({ where: { id } });
    if (!entity) throw new NotFound("Property not found");
    return entity;
  }

  public async create(body: CreatePropertyRequest, userId: number): Promise<PropertyEntity> { Show usages
    const owner : UserEntity = await this.userRepo.findOne({ where: { id: userId } });
    if (!owner) throw new BadRequest("Owner not found");
    const entity : PropertyEntity = this.repo.create({
      owner,
      title: body.title,
      description: body.description,
      rentType: body.rentType,
      price: body.price,
      status: body.status,
    });
    return await this.repo.save<PropertyEntity>(entity);
  }

  public async update(id: number, body: UpdatePropertyRequest): Promise<PropertyEntity> { Show usages
    const entity : PropertyEntity = await this.repo.findOne({ where: { id } });
    if (!entity) throw new NotFound("Property not found");
    Object.assign(entity, body);
    return await this.repo.save<PropertyEntity>(entity);
  }

  public async delete(id: number): Promise<void> { Show usages
    const res : DeleteResult = await this.repo.delete(id);
    if (!res.affected) throw new NotFound("Property not found");
  }
}

export default new PropertyService(); Show usages

```

Рисунок 3 – AdvertisementService

Вывод

В рамках работы был реализован RESTful API на основе реализованного ранее boilerplate.